

Infrastructure: Scalability & Performance

Document Redline & Search Service — System Design Reference

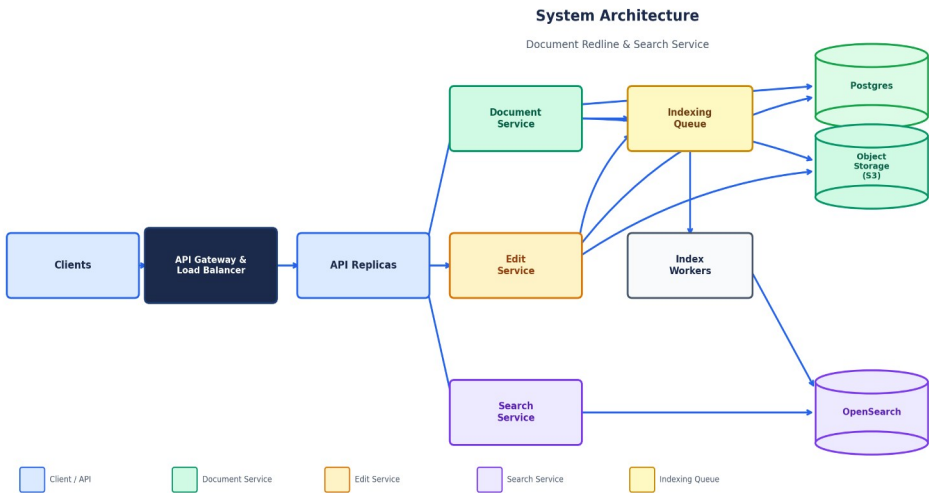
HOW I THINK ABOUT THIS SYSTEM

I broke the service into three distinct concerns, each with its own consistency model and scaling axis. Keeping these boundaries clean is the core design choice.

Service	Owns	Key Operation	Consistency Model
Document Service	Metadata + blob storage	Create, read, version	Strong (CP)
Edit Service	Redlines + concurrency control	Validate ETags, persist, enqueue	Strong (CP)
Search Service	Query results	Read from OpenSearch	Eventual (AP)

SYSTEM FLOW

At a high level, the request path looks like this:



SCALABILITY

API Layer

- Stateless replicas behind a load balancer — no in-process state
- Scales horizontally on CPU and request rate
- All persistent state lives in Postgres + S3

Write Path (synchronous, then async)

- Validate request (auth, ETag, size)

- 2. Persist metadata to Postgres and content to S3
- 3. Bump version and return response to caller
- 4. Enqueue indexing job (non-blocking)
- 5. Index workers update OpenSearch asynchronously

Key insight: *Writes never block on search indexing. Write latency is fully decoupled from OpenSearch throughput.*

Search Layer

- Workers consume the queue and update OpenSearch independently
- Shard indices by tenant or time bucket as volume grows
- Scale index workers separately from API replicas

Bottlenecks & Mitigations

Pressure Point	Mitigation Strategy
DB write throughput	Read replicas for dashboards; all version writes stay on primary
Hot tenants	Per-tenant rate limits, dedicated queues, or isolated index shards
Oversized documents	Streaming reads from S3, enforced size caps, chunked indexing pipeline

PERFORMANCE CHARACTERISTICS

All core operations run in $O(n)$ time or better:

- Replace by occurrence / range — $O(n)$ scan over document length
- Token index build — $O(n)$ one-pass tokenization and postings
- Phrase search (indexed) — $O(k)$ over candidate postings, not a full document scan

Production Optimizations

- Redis caching for hot documents and ETags to cut repeated GET + authz round-trips
- Server-side pagination caps on search to bound response size
- Async reindex keeps p99 latency for PATCH stable regardless of OpenSearch load
- Backpressure via queue depth alerts; shed load with 429s before overload cascades

KEY TRADE-OFF: INDEXING STRATEGY

Approach	Benefit	Cost
Synchronous reindex	Search reflects writes immediately	Higher write latency (p99 spikes)
Async reindex ✓ Chosen	Fast writes; stable p99 for PATCH	Search lags by seconds after write

I chose async because the write path is on the critical user path. Search freshness can tolerate a few seconds of lag in a document collaboration tool — that is an acceptable product trade-off.